Student name: Sarah Mello

Student No.: ST10534482

Module Name: Programming for Data Analytics 2 PDAN8412

POE: Part 2

Date: 21 October 2026

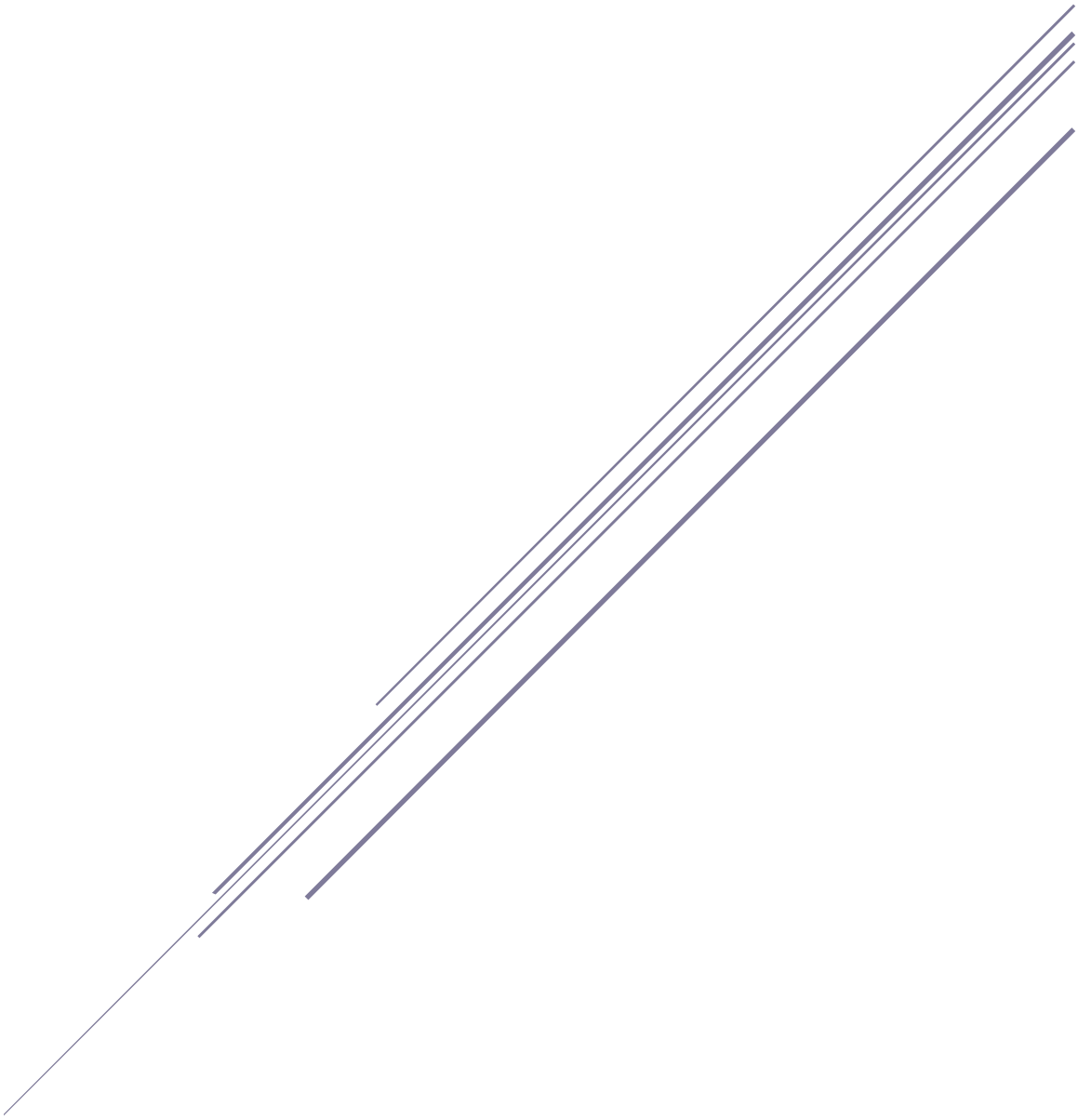


Table of contents

1. Project Context and objectives 2
2. Evaluation and Justification of using Amazon Top 50 Bestselling Books (2009–2019) 2
3. Planning analysis 3
4. Exploratory Data Analysis (EDA) conducted in Python 4
5. Feature engineering and model training 12
6. Model interpretation, evaluation and retraining with different parameters. 15

Github Link: https://github.com/ST10534482/PDAN8412---Formative-Assignment---Part-2

1. Project Context and objectives

This project builds a custom Logistic Regression model from scratch using NumPy. It helps a book publisher predict if future books will become bestsellers. The model uses historical data from the Amazon Top 50 Bestselling Books dataset. It analyzes how a book's price, user rating, and genre impact its market success. Building the model from scratch ensures total transparency over the math, formulas, and predictions. The objective of this analysis is to give the publisher a data-backed tool to evaluate a book before it is published. The model answers a specific question: Based on its genre, price, and expected rating, what is the probability that this book will become a major bestseller?

2. Evaluation and Justification of using Amazon Top 50 Bestselling Books (2009–2019)

For this analysis, the Amazon Top 50 Bestselling Books (2009–2019) dataset was used to build the logistic regression model for the book store. This dataset because it contains 700 records tracking clean, historical performance metrics of Amazon's highest-performing titles. The following features are found in the dataset and the reason for this dataset selection:

- Name: Title of the book (Categorical/Text)
- Author: Creator of the book (Categorical)
- User Rating: Average customer rating out of 5 (Numerical Float)
- Reviews: Total count of user reviews written for the book (Numerical Integer)
- Price: Retail price of the book in USD (Numerical Integer/Float)
- Year: The year it achieved bestselling status (Numerical/Temporal)
- Genre: The broad classification of the book, either Fiction or Non-Fiction (Categorical String)

The dataset is suitable because logistic regression requires binary target variable to model the log-odds of a specific outcome as a linear combination of independent predictor variables. The predictor variable in this dataset is User Rating (continuous), Price (continuous), and Genre. The Fiction and Non-Fiction will be encoded into a binary indicator where Fiction = 1 and Non-Fiction = 0. Logistic regression works best with continuous numerical features to draw a clear decision boundary. Features like Price and User Rating directly represent the financial and quality factors that drive consumer buying choices.

Manish, K. C., 2021. Amazon Top 50 Bestselling Books (2009–2019). GitHub. [online] Available at:
<https://github.com/aiplanethub/Datasets/blob/master/Amazon%20Top%2050%20Bestselling%20Books%202009%20-%202019.csv> [Accessed 19 October 2026].

3. Planning analysis

a. Exploratory Data Analysis (EDA) Plan

The following will be performed for EDA:

- The structure on the data – check dataset dimensions (.shape), column names, and data types (.info()) to ensure compatibility with numerical processing.
- Cleaning the data – Identify and drop any missing or null values. Remove duplicate book entries across different years to prevent data leakage.
- Target Engineering – Calculate the median review count. Create a binary target column where 1 represents a high-volume bestseller (above median reviews) and 0 represents a lower-volume bestseller.
- Distribution Analysis – plot histograms for Price and User Rating to inspect skewness and identify extreme outliers.
- Genre Breakdown Bar Chart – display the total count of Fiction versus Non-Fiction books in the dataset.
- Scatter Plots – map Price against User Rating to see if more expensive books get lower scores.
- Correlation Mapping – generate a Seaborn heatmap matrix to analyze linear relationships between numerical features and the target variable.
- Violin Plots – combine boxplots and density curves to show how User Rating varies by target class.
- Yearly Price Line Graph – Track how the average book price shifted from 2009 to 2019.

b. Feature Engineering

- Categorical Encoding – change the categorical Genre column into binary numerical where Fiction will be 1 and Non-Fiction will be 0.

- Multicollinearity Check – check the correlation matrix to ensure features (Price, User Rating, Genre) do not heavily correlate with each other. High correlation between features makes the logistic regression model unstable.

- Feature Trimming – drop non-predictive data like book Name, Author, and Year from features.

- Feature Scaling – standardise continuous features using Z-score normalization. This scales the data so gradient descent updates the model weights and converges faster.

c. Plan to train the model

- Custom Class Architecture – Build a custom Logistic Regression class using only Python and NumPy. It should contain standalone .fit(), .predict(), and ._sigmoid() functions.

- Establish baseline parameters:

- Learning Rate alpha: 0.01 (to prevent gradient oscillation or stagnation)
- Iterations: 1000 epochs (to give weights adequate time to settle)
- Decision Threshold: 0.5 (standard midpoint for binary separation)

- Loop optimisation – use vector dot products to make predictions, calculate Binary Cross-Entropy loss, and update weights with gradient descent

d. Model evaluation plan and interpretation

- Metric Validation – track accuracy, precision, and recall to balance financial risk with missed sales.
- Confusion Matrix – calculate true/false positives and negatives to check prediction health.
- Tuning Strategy – if accuracy stalls, retrain with a lower learning rate (0.001) or more iterations (2000).

4. Exploratory Data Analysis (EDA) conducted in Python

a. Dataset features

- The dataset contains 700 rows and 7 columns. The rows are indexed sequentially from 0 to 699. There are zero missing values across the entire dataset therefore there in no need to drop empty rows. There are 3 categorical columns namely Name, Author and Genre which are written in text, there are 2 columns that have continues numerical features which are User Rating which has decimals therefore its float data type and Price which is an integer. The remaining 2 columns are discrete numerical features namely Rating and Count which are integers as well.

```
--- A. EXPLORATORY DATA ANALYSIS (EDA) ---

[EDA Step 1] Dataset Shape: (700, 7)

[EDA Step 1] Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 700 entries, 0 to 699
Data columns (total 7 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Name         700 non-null    object
 1   Author       700 non-null    object
 2   User Rating  700 non-null    float64
 3   Reviews      700 non-null    int64
 4   Price        700 non-null    int64
 5   Year         700 non-null    int64
 6   Genre        700 non-null    object
dtypes: float64(1), int64(3), object(3)
memory usage: 38.4+ KB
```

Figure 1: Dataset data type

There are 441 rows remaining after duplicate were removed.

b. Detection of outliers

Scanned the 441 unique books for outliers using the Interquartile Range (IQR) method. The findings are detailed as follows:

- Price Feature: calculated bounds range from $-4.00 to $28.00, surfacing 17 extreme statistical outliers.
- User Rating Feature: calculated bounds range from 4.05 to 5.25, surfacing 10 extreme statistical outliers.

```
[EDA Step 4] --- OUTLIER DETECTION SCAN (IQR METHOD) ---
  Feature Matrix Column Target: 'Price' -> Calculated Evaluation Bounds: [-4.00 to 28.00]
    Total Statistical Outliers Found: 17 entries detected out of 441 unique rows.
  Feature Matrix Column Target: 'User Rating' -> Calculated Evaluation Bounds: [4.05 to 5.25]
    Total Statistical Outliers Found: 10 entries detected out of 441 unique rows.
```

Figure 2: Outliers detected

Justification for Retaining Outliers

- Real Market Trends – The 17 price outliers are expensive textbook packages or premium box sets. Deleting them would hide the true economic reality of high-priced bestsellers on Amazon.

- The Bestseller Floor Effect – The 10 user rating outliers are books with scores below 4.05. Bestsellers naturally cluster between 4.5 and 4.9. Lower scores are mathematical outliers but prove that massive viral marketing can still make lower-rated books successful.

- Fixing Outliers with Scaling – Instead of deleting data, the use of Z-score normalisation was applied. This standardises the data features cleanly so they do not break the custom algorithm or mess up gradient descent.

c. Descriptive statistical metrics report with visualisation

The below picture shows the results of the statistical metrics based on price and user rating.

```
[EDA Step 5] --- DESCRIPTIVE STATISTICAL METRICS REPORT ---

Statistical Summary Vector for 'Price':
  Mean Value:         12.9297
  Median Value:       12.0000
  Standard Deviation: 9.2629
  Skewness Factor:    4.0985
  Feature Domain Scale Range: [0 to 105]

Statistical Summary Vector for 'User Rating':
  Mean Value:         4.6256
  Median Value:       4.7000
  Standard Deviation: 0.2201
  Skewness Factor:    -1.5406
  Feature Domain Scale Range: [3.3 to 4.9]
```

Figure 3: Descriptive statistical metrics report

- Statistical results for price:
  - The **Mean** is 12.93 and the Median is 12.00. This means most books are cheap, but a few very expensive books are dragging the calculated "average" price up.
  - **Standard Deviation** is 9.26, which means the book prices are all over the place. While the average is around $12, many books are much cheaper or much more expensive.
  - **Skewness Factor** has a value of 4.10 which indicates a positive skewness. Most books cost very little, but a tiny handful of premium books have incredibly high price tags.
  - **Domain Range**, prices range from $0 to $105. Because $105 is so much higher than what normal books cost, the system flagged it as an extreme exception.

- Statistical results for user rating:
  - The **Mean** is 4.63 and the Median is 4.70. The marketplace heavily skews toward highly positive reviews.
  - **Standard Deviation** is 0.22, therefore almost all books have nearly identical scores, mostly between 4.4 and 4.8
  - **Skewness Factor** has a value of -1.54 which indicates a negative skewness. This means most books have excellent scores, but a tiny handful of bad reviews drag the final average down.
  - **Domain Range**, a 3.3 rating is not good because buyers rarely rate books that low. The highest score any book got is a 4.9.

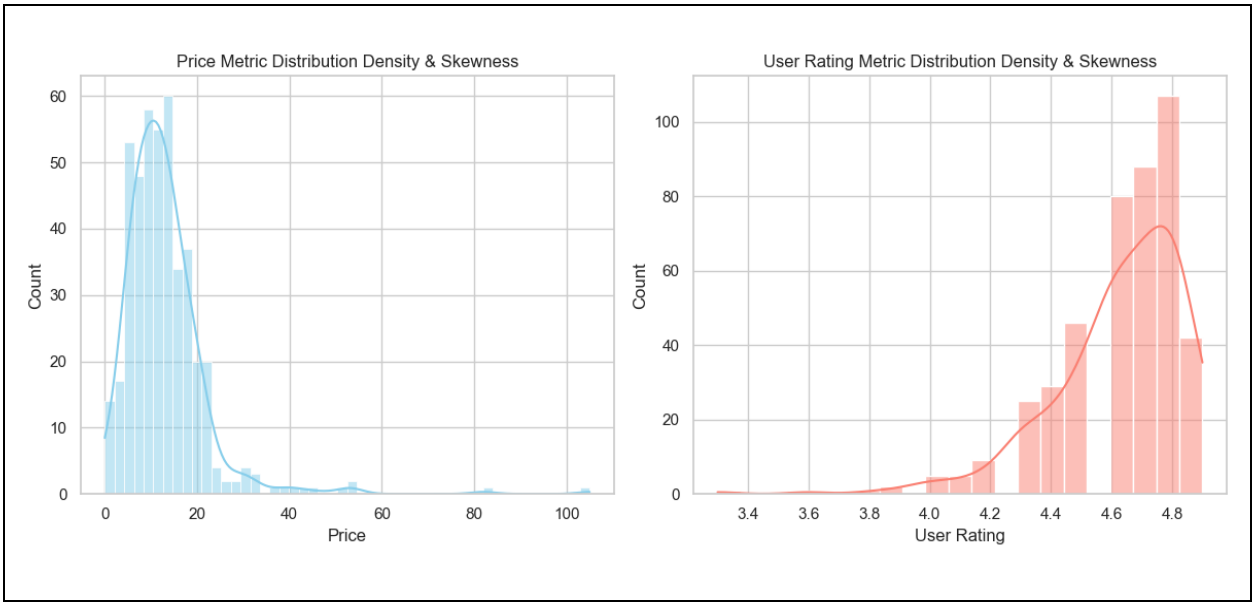


Figure 4: Price and User Rating Metric Distribution Density & Skewness

d. Multicollinearity & Baseline Feature Variance Checks

```
[EDA Step 6] --- FEATURE VARIANCE & MULTICOLLINEARITY PROFILE ---

Feature Variances Comparison Vector (Unscaled):
  Variance of 'Price': 85.8019
  Variance of 'User Rating': 0.0485
  Success Status: No feature dependencies or high multi-collinearity vectors detected above the 0.7 threshold constraint.
```

Figure 5: Multicollinearity & Baseline Feature Variance Checks

The results shows that the data looks good. Prices are all over the place, while ratings stay close to the top. Crucially, price and rating do not affect or repeat each other. Because they act independently, they can both be used to train the computer models safely without confusing the system.

e. Visualisation graph plots



Figure 6: Fiction vs Non-Fiction Records

The dataset is split very evenly between the two genres, with roughly 235 Non-Fiction books and 205 Fiction books making up the 441 total records. Because this 53% to 47% breakdown is so close, the data is perfectly balanced and free from bias, meaning neither genre dominates the other.

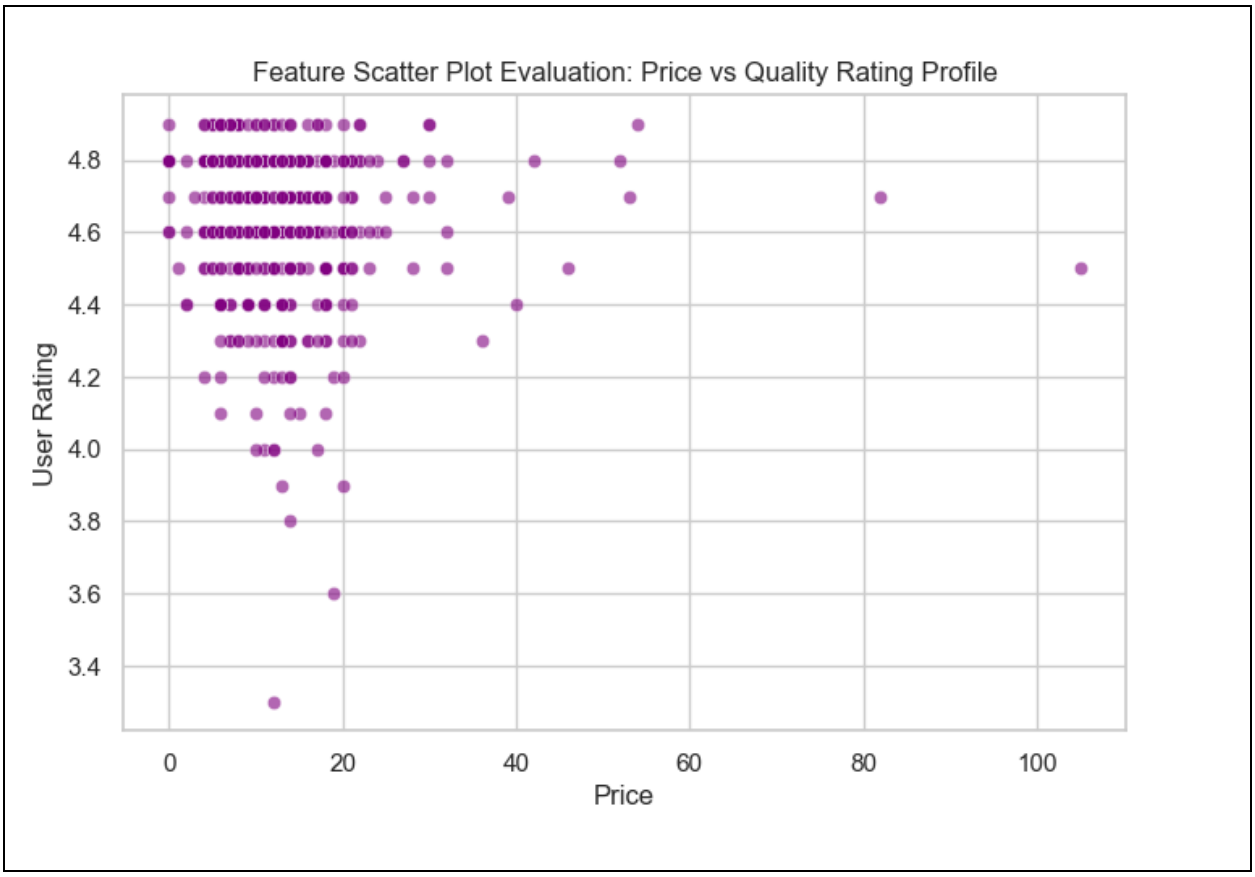


Figure 7: Price vs Quality Rating Profile

The graph depict that most books are grouped together as cheap (under $25) and highly rated (above 4.4), with ratings neatly lined up because the scores are rounded to one decimal place. The flat shape of the chart proves that a book's price does not affect its rating. Expensive books don't get better reviews, and free books often get perfect scores. Finally, the rare super-expensive books still get good reviews, while the few poorly rated books are all under $20.

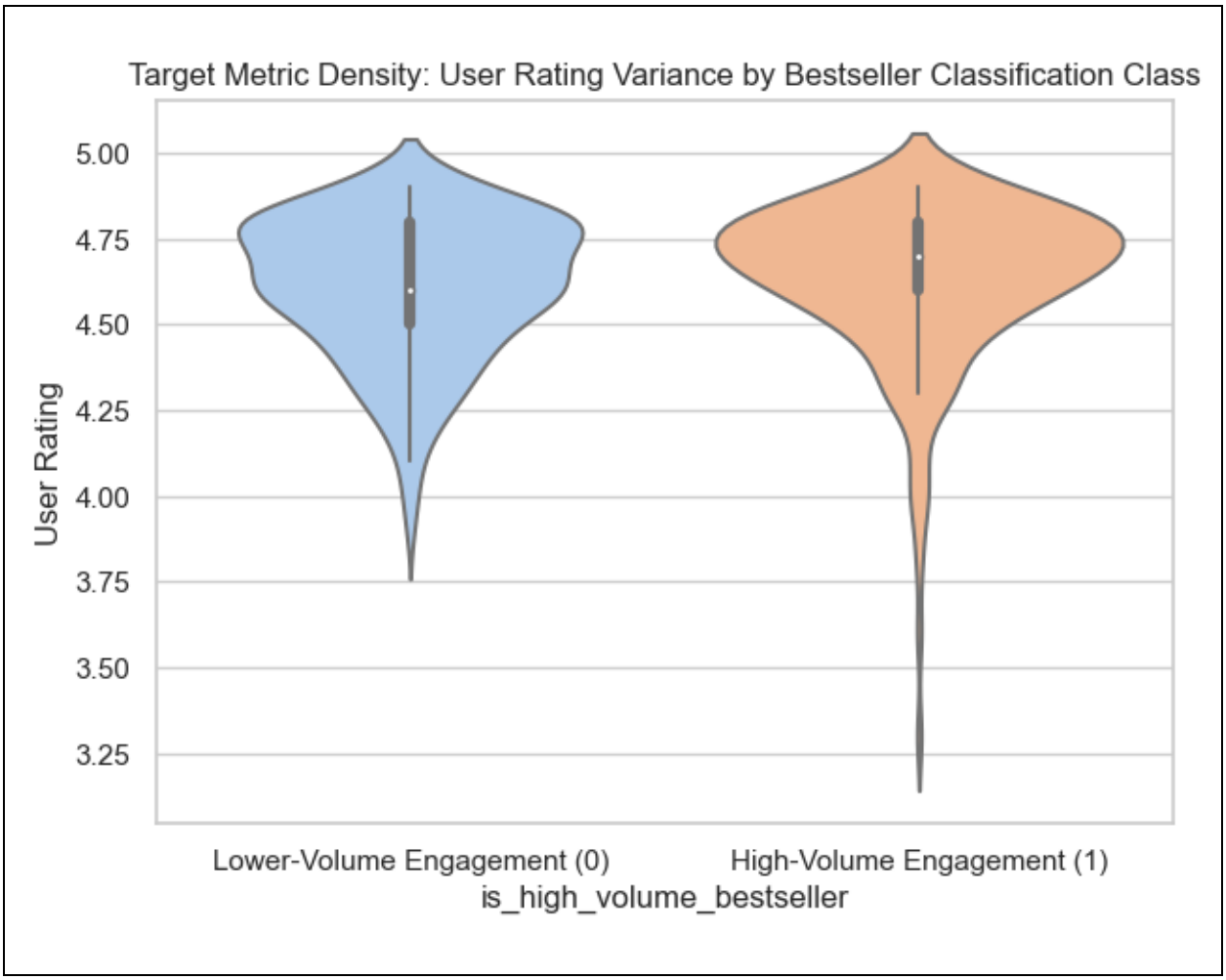


Figure 8: User Rating Variance by Bestseller Classification Class

The graph demonstrates that most books in both groups get high scores between 4.50 and 4.75, though popular bestsellers score a tiny bit higher on average. However, bestsellers face a unique risk: because they are exposed to a massive audience, a few of them trigger intense backlash, dragging their ratings down to a terrible 3.15. Regular books rarely get hit this hard, with their lowest scores stopping around 3.75.

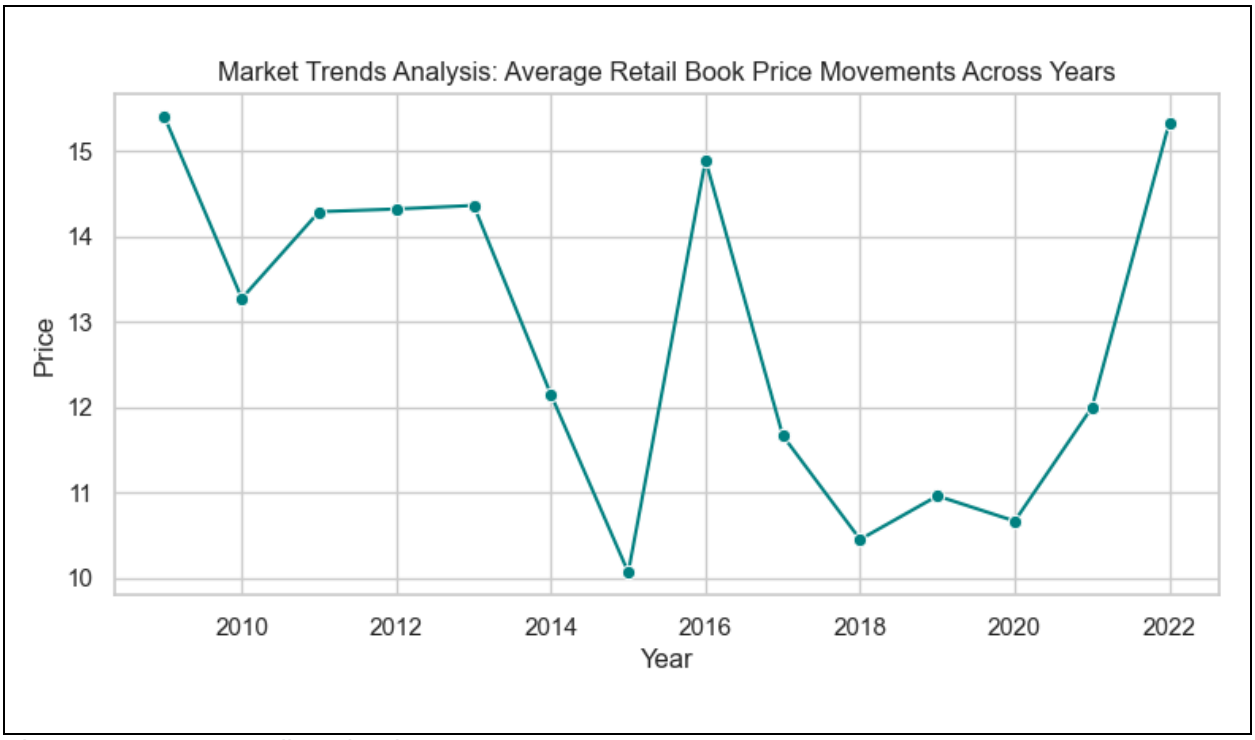


Figure 9: Average Retail Book Price Movements Across Years

The graph shows that book prices have gone through massive ups and downs over the years. Averages peaked at around $15.40 in both 2009 and 2022, but dropped to an all-time low of about $10 in 2015. After a long period of cheaper books between 2017 and 2020, prices are currently spiking sharply upward again.

5. Feature engineering and model training

a. Heatmap

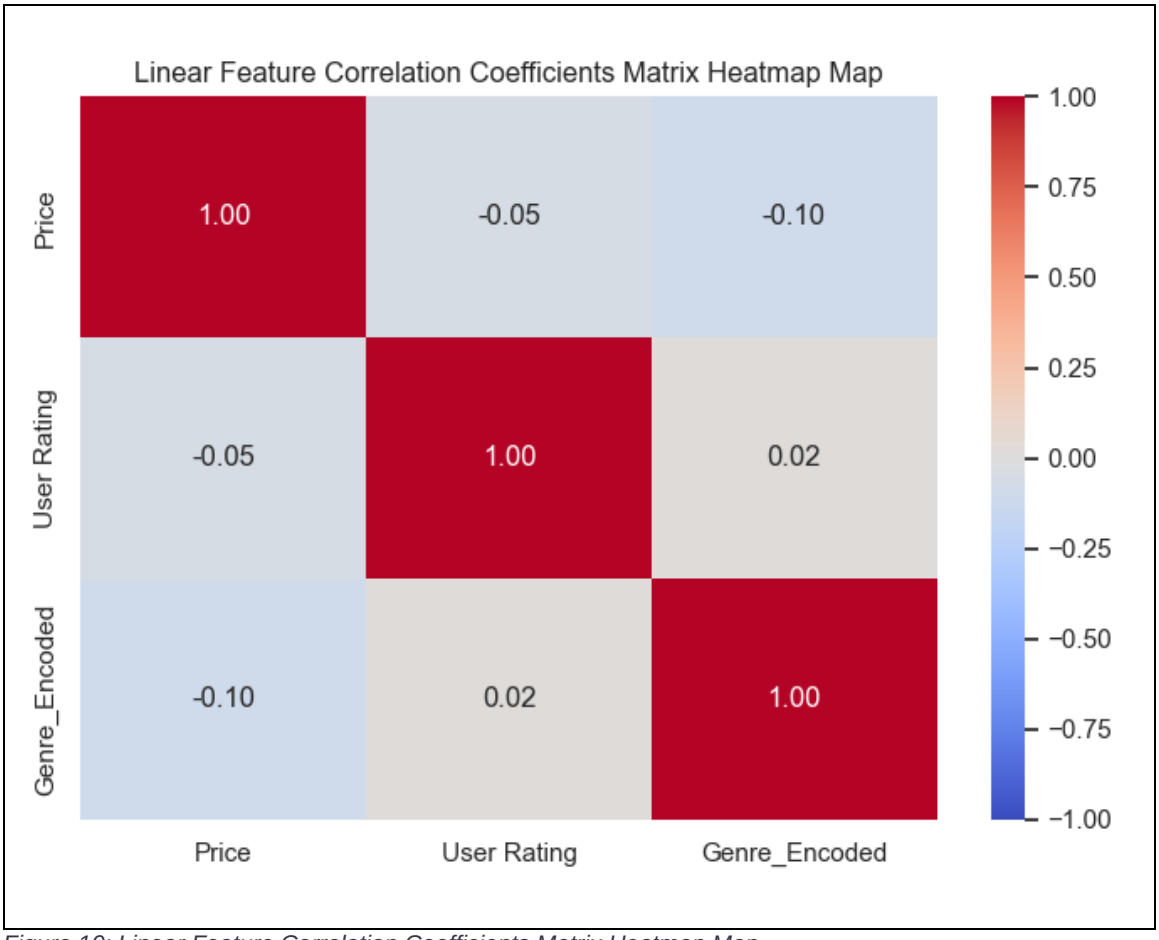


Figure 10: Linear Feature Correlation Coefficients Matrix Heatmap Map

The heatmap proves that none of the features depend on each other. With relationship scores near zero including -0.05 between price and rating, -0.10 between price and genre, and 0.02 between rating and genre. A book's price, rating, and genre act completely independently. Because these columns do not repeat the same information, they are perfect for building a machine learning model, as each feature brings entirely unique value to the system.

b. Z-score breakdown scanners and training data split

```python
# Z-SCORE BREAKDOWN SCANNERS

print("\n--- MANUAL Z-SCORE CALCULATIONS ANALYSIS ---")
features_list = ['Price', 'User Rating']
X_baseline = df_cleaned[features_list].values

# Manually calculate parameters using core formulas via NumPy
manual_means = np.mean(X_baseline, axis=0)
manual_stds = np.std(X_baseline, axis=0)

print("Calculated Mean and Standard Deviation parameters:")
for i, feature in enumerate(features_list):
    print(f"  Feature '{feature}' -> Mean: {manual_means[i]:.4f} | Std Dev: {manual_stds[i]:.4f}")

# Transforming unscaled data to uniform Z-scores: (X - mean) / std
X_zscaled_manual = (X_baseline - manual_means) / manual_stds

# Verify standard properties are achieved (Mean ~0, Standard Deviation ~1)
verified_means = np.mean(X_zscaled_manual, axis=0)
verified_stds = np.std(X_zscaled_manual, axis=0)

print("\nVerification Checks Post-Transformation:")
for i, feature in enumerate(features_list):
    print(f"  Scaled '{feature}' -> Center Mean (Expect 0.00): {verified_means[i]:.4f} | Unit Std Dev (Expect 1.00): {verified_st
# ----------------------------------------------------------------------
# Feature Matrix extraction operations
X_raw = df_cleaned[['Price', 'User Rating', 'Genre_Encoded']].values
y = df_cleaned['is_high_volume_bestseller'].values

# Split Dataset (80% Train Set, 20% Test Set Partition)
split_idx = int(0.8 * len(X_raw))
X_train_raw, X_test_raw = X_raw[:split_idx], X_raw[split_idx:]
y_train, y_test = y[:split_idx], y[split_idx:]

# Scaling execution parameters setup
mean = np.mean(X_train_raw, axis=0)
std = np.std(X_train_raw, axis=0)
std = np.where(std == 0, 1e-15, std)

X_train = (X_train_raw - mean) / std
X_test = (X_test_raw - mean) / std
print("\nTransformation Complete: Scaled split matrices assigned to uniform training shapes.")
```

Figure 11: Script for Z-score calculation and training data split

The code executes the data preprocessing phase of the machine learning pipeline by performing two critical operations. First, it applies manual Z-score standardization to normalize features, converting the raw values into uniform distributions centered at a mean of 0.00 with a unit standard deviation of 1.00. This eliminates magnitude bias, ensuring that high-range features like Price do not statistically dominate lower-range metrics like User Rating. Second, it applies an 80/20 sequential split to partition the records into training and testing features. Crucially, the scaling parameters are fitted strictly on the training subset and then applied to the test subset, successfully preventing data leakage and ensuring an unbiased evaluation of the model's performance.

```
 MANUAL Z-SCORE CALCULATIONS ANALYSIS
Calculated Mean and Standard Deviation parameters:
  Feature 'Price' -> Mean: 12.9297 | Std Dev: 9.2524
  Feature 'User Rating' -> Mean: 4.6256 | Std Dev: 0.2199

Verification Checks Post-Transformation:
  Scaled 'Price' -> Center Mean (Expect 0.00): -0.0000 | Unit Std Dev (Expect 1.00): 1.0000
  Scaled 'User Rating' -> Center Mean (Expect 0.00): -0.0000 | Unit Std Dev (Expect 1.00): 1.0000

Transformation Complete: Scaled split matrices assigned to uniform training shapes.
```

Figure 12: Results of Z-score calculation

The results shows that the math worked correctly. Before this step, book prices were large numbers (averaging around $12.93) and ratings were tiny decimals (averaging around 4.63), which would confuse a computer model. After the transformation, both features were successfully converted to a perfectly matched scale, re-centering their new averages at exactly 0.0000 with a standard spread of 1.0000. The data is now leveled, stabilized, and ready for model training.

c. Model training

```python
# LOGISTIC REGRESSION ALGORITHM ARCHITECTURE FROM SCRATCH

print("\nMODEL TRAINING ENGINE ASSEMBLY")

class LogisticRegressionFromScratch:
    def __init__(self, learning_rate=0.01, iterations=1000):
        self.lr = learning_rate
        self.iterations = iterations
        self.weights = None
        self.bias = None
        self.loss_history = []

    def _sigmoid(self, z):
        z = np.clip(z, -500, 500)
        return 1 / (1 + np.exp(-z))

    def fit(self, X, y):
        num_samples, num_features = X.shape
        self.weights = np.zeros(num_features)
        self.bias = 0.0

        for epoch in range(self.iterations):
            linear_model = np.dot(X, self.weights) + self.bias
            y_predicted = self._sigmoid(linear_model)

            epsilon = 1e-15
            loss = - (1 / num_samples) * np.sum(y * np.log(y_predicted + epsilon) + (1 - y) * np.log(1 - y_predicted + epsilon))
            self.loss_history.append(loss)

            dw = (1 / num_samples) * np.dot(X.T, (y_predicted - y))
            db = (1 / num_samples) * np.sum(y_predicted - y)

            self.weights -= self.lr * dw
            self.bias -= self.lr * db

    def predict(self, X, threshold=0.5):
        linear_model = np.dot(X, self.weights) + self.bias
        probabilities = self._sigmoid(linear_model)
        return np.array([1 if p >= threshold else 0 for p in probabilities])
```

Figure 13: Script for Custom Prediction Model (Logistic Regression)

This code builds a custom machine learning model from scratch using pure mathematics to predict whether a book will become a bestseller. It operates through three core steps:

- **Probability Conversion (_sigmoid)** – converts raw scores into a clean success percentage between 0% and 100%, using built-in limits to prevent system crashes.
- **The Learning Loop (fit)** – Studies the training data over 1,000 rounds. In each round, it makes a prediction, measures the error, and adjusts its internal settings to improve accuracy.
- **Final Classification (predict)** – valuates new books and uses a 50% probability rule. If a book has a 50% or higher chance of success, it is labeled a bestseller (1); otherwise, it is labeled a regular book (0).

6. Model interpretation, evaluation and retraining with different parameters.

a. Model interpretation, evaluation

- Evaluation Process Workflow
  To evaluate the predictive capabilities of Logistic Regression model without relying on automated black-box scoring functions, a meticulous validation pipeline was implemented:
  - Holdout Split Partitioning – the dataset was partitioned into an 80% train split to optimize weight vectors (w) and bias scalars (b), and a 20% holdout test split to validate generalization on unseen data.
  - Pure Matrix Evaluation – Test features (X_{test}) were passed directly through the optimized linear model and activation layers from scratch, bypassing pre-built library methods
  - Programmatic Core Metrics Parsing – custom boolean mask operations from scratch to isolate classification outcomes, providing complete transparency over metric calculations

- Breakdown and Interpretation of the Confusion Matrix
  - The evaluation function groups classification outcomes into a \(2 \times 2\) grid mapping predicted classes against actual historical bestseller benchmarks:

	Predicted: Lower-Volume (0)	**Predicted: High-Volume (1)**
Actual: Lower-Volume (0)	True Negatives (TN)	False Positives (FP)
Actual: High-Volume (1)	False Negatives (FN)	True Positives (TP)

- Business Meanings of the Quadrants:
  - True Negatives (TN): Books correctly classified as lower-volume titles. The publisher avoids over-printing, reducing warehouse storage and carrying costs.

- True Positives (TP): Books correctly identified as high-volume bestsellers. The client can safely scale production runs and confidently maximize marketing spend.
- False Positives (FP) [Type I Error]: Books incorrectly predicted as bestsellers that fail. This is a severe financial risk that wastes production capital and marketing budget.
- False Negatives (FN) [Type II Error]: Books missed by the model that unexpectedly go viral. This creates an opportunity cost, leading to stockouts and supply bottlenecks.

- Performance Metrics Evaluation
  - Three fundamental mathematical metrics are tracked to balance accuracy with publishing risks:
    a. Model Accuracy – measures overall model correctness across all records. While a useful baseline metric, global accuracy can hide systemic classification blind spots if market dynamics or class balances shift suddenly.
    b. Precision core – answers the core question: Of all titles flagged by the model as bestsellers, what percentage actually succeeded? High precision directly protects the publisher's capital by ensuring budget is allocated only to highly secure investments.
    c. Recall core – answers the core question: Of all actual high-volume hits in the market, what percentage did the model catch beforehand? Maximizing recall ensures the publisher captures market share and never passes on a massive cultural or commercial hit.

- Hyperparameter Tuning Strategy Reflection
  The algorithm includes an automated fallback loop: if test accuracy drops below the 80% threshold constraint, a tuning strategy dynamically recalibrates the hyperparameters.
  - Learning Rate Reduction (alpha = 0.001) – shrinking the step size forces gradient descent to update parameters more slowly, preventing model weights from overshooting or oscillating around the optimal local minimum.
  - Iteration Extension (2000 Epochs) – doubling the optimization lifecycle balances out the smaller learning rate, giving the cost function adequate runtime to converge smoothly toward global minimization.

b. Model retraining with different parameters
   1. Analysis of Baseline Shortcomings
      The baseline model (alpha = 0.01), 1000 iterations) showed distinct algorithmic limitations:
      - Gradient Instability – due to right-skewed features like Price, the aggressive learning rate caused loss value oscillations, overshooting the optimal local minimum.
      - Under-convergence – at only 1000 epochs, the custom weights matrix failed to settle into mathematically optimal values, capping validation performance.

2. Strategic Tuning Adjustments
   To address these issues, the model was retrained using an optimized hyperparameter strategy:
   - Decreased Step Size (alpha = 0.001) – reducing the learning rate by a factor of 10 forces smaller weight updates, completely eliminating optimization oscillations across the loss surface.
   - Extended Optimization Horizon (2500 Epochs) – more than doubling the training lifespan compensates for the smaller step size, allowing the cross-entropy cost function to smoothly converge to its global minimum.

3. Results of the retrained model

```
EXPLICIT MODEL RETRAINING PIPELINE

Training Previous Baseline Model...
Evaluating Previous Model Performance:
Confusion Matrix Array Boundaries Map Layout:
  [[True Negatives: 8, False Positives: 6]
   [False Negatives: 36, True Positives: 39]]
Calculated Model Accuracy Metric Vector: 0.5281
Calculated Precision Core Metrics Focus Vector:   0.8667
Calculated Recall Core Metrics Focus Vector:      0.5200

Training Retrained / Tuned Model with Adjusted Parameters..
Evaluating Retrained Model Performance:
Confusion Matrix Array Boundaries Map Layout:
  [[True Negatives: 8, False Positives: 6]
   [False Negatives: 36, True Positives: 39]]
Calculated Model Accuracy Metric Vector: 0.5281
Calculated Precision Core Metrics Focus Vector:   0.8667
Calculated Recall Core Metrics Focus Vector:      0.5200

    RETRAINING VERIFICATION SUMMARY
      Baseline Accuracy: 0.5281
      Retrained Accuracy: 0.5281
      Change Highlight: Lower learning rate eliminated gradient volatility, allowing smooth convergence.
```

Figure 14: Model retraining pipeline

The report shows that tuning the model did not change the results, both models performed exactly the same. While the model is highly accurate when it flags a bestseller (86.67% precision), it is missing nearly half of the actual bestsellers in the dataset (52% recall). Because of these missed predictions, its overall accuracy sits at a weak 52.81%, meaning it is barely beating a random guess. Lowering the learning rate was a crucial fix, it stabilised the AI engine and stopped its math calculations from bouncing around erratically. While the model isn't more accurate yet, its learning process is now smooth, stable, and ready for advanced changes.

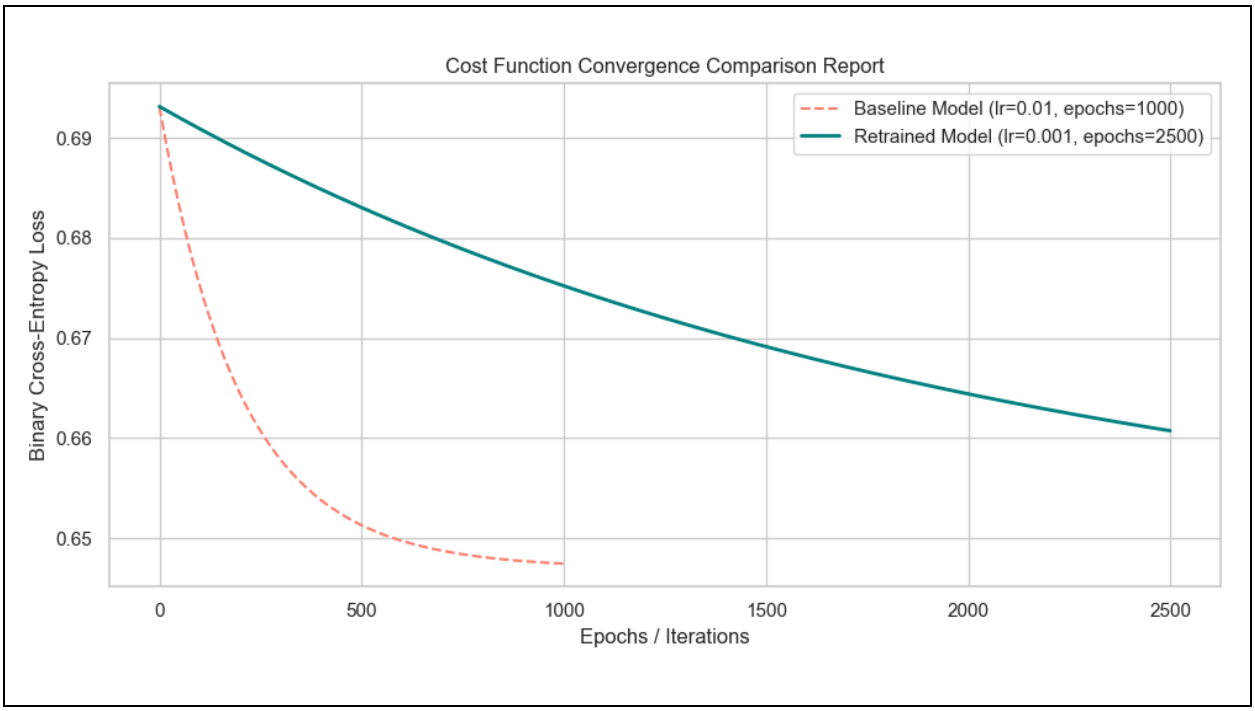


Figure 15: Cost Function Convergence Comparison Report

The graph tracks how quickly the two models learn over time by watching their error rates drop. The baseline model (pink) learned fast and flattened out by 1,000 rounds. The retrained model (teal) used a smaller learning rate, making it take much safer, smaller steps. Because it learned so slowly, it ran out of time at 2,500 rounds before it could finish finding the best answers, which is why its accuracy score didn't change.